

Practical Step-by-Step Example: MLP Forward, Backward & Adam Update

Reference Material for Defense
Multilayer Perceptron Project

September 11, 2026

Contents

1	Architecture & Initial Setup	2
2	Step 1: Forward Pass	3
2.1	Layer 1: Pre-activations Z^1 and Activations A^1	3
2.2	Layer 2: Pre-activations Z^2 and Output A^2	4
2.3	Loss Computation (Categorical Cross-Entropy)	4
3	Step 2: Backward Pass & Chain Rule	6
3.1	Proof: Simplification of Softmax with Categorical Cross-Entropy	6
3.2	Proof: Derivation of Error Propagation (δ^l) and Bias Gradient ($\frac{\partial L}{\partial b^l}$)	7
3.3	Vectorization and Matrix Transposition	8
3.4	Output Layer Gradient (δ^2)	8
3.5	Weight and Bias Gradients at Layer 2 (dW^2, db^2)	8
3.6	Hidden Layer Error Projection (δ^1)	9
3.7	Weight and Bias Gradients at Layer 1 (dW^1, db^1)	9
4	Step 3: Adam Optimizer Step	10
4.1	Accumulation & Bias Correction Formulas	10
4.2	Explicit Parameter Updates	10
4.2.1	Update of Layer 1 Weights (W^1)	10
4.2.2	Update of Layer 1 Biases (b^1)	11
4.2.3	Update of Layer 2 Weights (W^2)	11
4.2.4	Update of Layer 2 Biases (b^2)	12
4.3	Summary of Parameter Updates	12
4.4	Behavior in Subsequent Iterations ($t \gg 1$)	12

1 Architecture & Initial Setup

Network Topology

- **Input Layer** ($l = 0$): $n_0 = 4$ features.
- **Hidden Layer** ($l = 1$): $n_1 = 3$ neurons (Activation: ReLU).
- **Output Layer** ($l = 2$): $n_2 = 2$ neurons (Activation: Softmax).
- **Batch Size** (m): 2 normalized samples.

Initial Data Matrices

Input Matrix $X = A^0$ (4×2):

$$X = A^0 = \begin{bmatrix} 0.5 & -0.2 \\ 0.8 & 0.4 \\ -0.1 & 0.9 \\ 0.3 & -0.6 \end{bmatrix}$$

We consider two samples, each with four features, in order to illustrate the complete process using a small mini-batch. The process for a single sample would be analogous, starting with A^0 represented as a 4×1 column vector.

Layer 1 Parameters ($W^1 \in \mathbb{R}^{3 \times 4}$, $b^1 \in \mathbb{R}^{3 \times 1}$):

$$W^1 = \begin{bmatrix} 0.35 & -0.50 & 0.20 & -0.10 \\ -0.25 & 0.40 & -0.30 & 0.15 \\ 0.10 & 0.05 & 0.60 & -0.45 \end{bmatrix}, \quad b^1 = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

Layer 2 Parameters ($W^2 \in \mathbb{R}^{2 \times 3}$, $b^2 \in \mathbb{R}^{2 \times 1}$):

$$W^2 = \begin{bmatrix} 0.40 & -0.30 & 0.50 \\ -0.10 & 0.60 & -0.20 \end{bmatrix}, \quad b^2 = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Target Labels Y (One-Hot, 2×2):

$$Y = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

2 Step 1: Forward Pass

2.1 Layer 1: Pre-activations Z^1 and Activations A^1

To process the mini-batch of size $m = 2$, we compute the matrix multiplication $W^1 A^0$ and add the bias vector b^1 via broadcasting:

$$Z^1 = W^1 A^0 + b^1$$

Substituting the numerical matrices:

$$Z^1 = \begin{bmatrix} 0.35 & -0.50 & 0.20 & -0.10 \\ -0.25 & 0.40 & -0.30 & 0.15 \\ 0.10 & 0.05 & 0.60 & -0.45 \end{bmatrix} \begin{bmatrix} 0.5 & -0.2 \\ 0.8 & 0.4 \\ -0.1 & 0.9 \\ 0.3 & -0.6 \end{bmatrix} + \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

Evaluating each element $(Z^1)_{ij}$:

Sample 1 (Column 1):

$$Z_{11}^1 = (0.35)(0.5) + (-0.50)(0.8) + (0.20)(-0.1) + (-0.10)(0.3) = -0.275$$

$$Z_{21}^1 = (-0.25)(0.5) + (0.40)(0.8) + (-0.30)(-0.1) + (0.15)(0.3) = 0.270$$

$$Z_{31}^1 = (0.10)(0.5) + (0.05)(0.8) + (0.60)(-0.1) + (-0.45)(0.3) = -0.105$$

Sample 2 (Column 2):

$$Z_{12}^1 = (0.35)(-0.2) + (-0.50)(0.4) + (0.20)(0.9) + (-0.10)(-0.6) = -0.030$$

$$Z_{22}^1 = (-0.25)(-0.2) + (0.40)(0.4) + (-0.30)(0.9) + (0.15)(-0.6) = -0.150$$

$$Z_{32}^1 = (0.10)(-0.2) + (0.05)(0.4) + (0.60)(0.9) + (-0.45)(-0.6) = 0.810$$

Therefore,

$$Z^1 = \begin{bmatrix} -0.275 & -0.030 \\ 0.270 & -0.150 \\ -0.105 & 0.810 \end{bmatrix}$$

Next, we apply the element-wise Rectified Linear Unit activation function:

$$\text{ReLU}(z) = \max(0, z)$$

Thus,

$$A^1 = \begin{bmatrix} \max(0, -0.275) & \max(0, -0.030) \\ \max(0, 0.270) & \max(0, -0.150) \\ \max(0, -0.105) & \max(0, 0.810) \end{bmatrix}$$

and therefore,

$$A^1 = \begin{bmatrix} 0.000 & 0.000 \\ 0.270 & 0.000 \\ 0.000 & 0.810 \end{bmatrix}$$

Notice how ReLU deactivates negative inputs by setting them to zero. This produces a sparse representation in the hidden layer. In this example, the non-zero activations are A_{21}^1 and A_{32}^1 .

2.2 Layer 2: Pre-activations Z^2 and Output A^2

The second layer receives the hidden-layer activations A^1 as input. The pre-activation matrix is computed as

$$Z^2 = W^2 A^1 + b^2$$

Substituting the numerical matrices:

$$Z^2 = \begin{bmatrix} 0.40 & -0.30 & 0.50 \\ -0.10 & 0.60 & -0.20 \end{bmatrix} \begin{bmatrix} 0.000 & 0.000 \\ 0.270 & 0.000 \\ 0.000 & 0.810 \end{bmatrix} + \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Evaluating each element:

$$Z_{11}^2 = (0.40)(0.000) + (-0.30)(0.270) + (0.50)(0.000) = -0.081$$

$$Z_{21}^2 = (-0.10)(0.000) + (0.60)(0.270) + (-0.20)(0.000) = 0.162$$

$$Z_{12}^2 = (0.40)(0.000) + (-0.30)(0.000) + (0.50)(0.810) = 0.405$$

$$Z_{22}^2 = (-0.10)(0.000) + (0.60)(0.000) + (-0.20)(0.810) = -0.162$$

Therefore,

$$Z^2 = \begin{bmatrix} -0.081 & 0.405 \\ 0.162 & -0.162 \end{bmatrix}$$

The output layer uses the Softmax activation function. For each sample (column), Softmax converts the logits into probabilities:

$$A_{ki}^2 = \frac{\exp(Z_{ki}^2)}{\sum_{j=1}^{n_2} \exp(Z_{ji}^2)}$$

For Sample 1:

$$A_{11}^2 = \frac{e^{-0.081}}{e^{-0.081} + e^{0.162}} \approx 0.4395, \quad A_{21}^2 = \frac{e^{0.162}}{e^{-0.081} + e^{0.162}} \approx 0.5605$$

For Sample 2:

$$A_{12}^2 = \frac{e^{0.405}}{e^{0.405} + e^{-0.162}} \approx 0.6381, \quad A_{22}^2 = \frac{e^{-0.162}}{e^{0.405} + e^{-0.162}} \approx 0.3619$$

Thus,

$$A^2 \approx \begin{bmatrix} 0.4395 & 0.6381 \\ 0.5605 & 0.3619 \end{bmatrix}$$

Each column sums to approximately 1, as expected for a Softmax output.

2.3 Loss Computation (Categorical Cross-Entropy)

For a mini-batch of $m = 2$ samples, the categorical cross-entropy loss is

$$L = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^{n_2} Y_{ki} \ln(A_{ki}^2)$$

Since the target labels are one-hot encoded,

$$Y = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

the loss becomes

$$L = -\frac{1}{2} [\ln(A_{11}^2) + \ln(A_{22}^2)]$$

Using the numerical values,

$$L = -\frac{1}{2} [\ln(0.4395) + \ln(0.3619)] = -\frac{1}{2} [(-0.8220) + (-1.0163)] = -\frac{1}{2}(-1.8383)$$

Therefore, the initial mini-batch loss is

$$\boxed{L \approx 0.9192}$$

3 Step 2: Backward Pass & Chain Rule

3.1 Proof: Simplification of Softmax with Categorical Cross-Entropy

A common question during defense is why the gradient at the output layer simplifies to the straightforward subtraction $\delta^2 = A^2 - Y$.

Let z_i be the i -th pre-activation logit, $a_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$ the Softmax output, and Y_k the true one-hot target.

1. Derivative of Softmax $\left(\frac{\partial a_k}{\partial z_i}\right)$

Using the quotient rule on the Softmax definition:

- Case $k = i$: $\frac{\partial a_i}{\partial z_i} = a_i(1 - a_i)$
- Case $k \neq i$: $\frac{\partial a_k}{\partial z_i} = -a_k a_i$

Combining both cases using the Kronecker delta δ_{ki} :

$$\frac{\partial a_k}{\partial z_i} = a_k(\delta_{ki} - a_i)$$

2. Applying the Chain Rule for Loss L

The Categorical Cross-Entropy loss for a single sample is $L = -\sum_k Y_k \ln(a_k)$. By applying the chain rule to compute $\delta_i = \frac{\partial L}{\partial z_i}$:

$$\delta_i = \sum_k \frac{\partial L}{\partial a_k} \frac{\partial a_k}{\partial z_i}$$

Since $\frac{\partial L}{\partial a_k} = -\frac{Y_k}{a_k}$:

$$\delta_i = \sum_k \left(-\frac{Y_k}{a_k}\right) [a_k(\delta_{ki} - a_i)] = -\sum_k Y_k(\delta_{ki} - a_i) = -\sum_k Y_k \delta_{ki} + \sum_k Y_k a_i$$

3. Final Reduction

1. Since Y is a one-hot vector, $\sum_k Y_k = 1$. Therefore, $\sum_k Y_k a_i = a_i \sum_k Y_k = a_i$.
2. The Kronecker delta δ_{ki} isolates the term where $k = i$, so $\sum_k Y_k \delta_{ki} = Y_i$.

Substituting these two reductions back into the expression:

$$\delta_i = a_i - Y_i \implies \boxed{\delta^2 = A^2 - Y}$$

This elegant cancellation eliminates numerical instability during training and speeds up computation.

3.2 Proof: Derivation of Error Propagation (δ^l) and Bias Gradient ($\frac{\partial L}{\partial b^l}$)

To understand how errors flow backward through hidden layers, we apply the multivariable chain rule to the fundamental forward pass equations of any layer l :

$$z^l = W^l a^{l-1} + b^l, \quad a^l = \sigma(z^l)$$

where the local error vector is formally defined as $\delta^l = \frac{\partial L}{\partial z^l}$.

1. Derivation of the Bias Gradient (BP3)

We compute the partial derivative of the loss function L with respect to the bias vector b^l . Applying the chain rule:

$$\frac{\partial L}{\partial b^l} = \frac{\partial L}{\partial z^l} \cdot \frac{\partial z^l}{\partial b^l}$$

From the pre-activation equation $z^l = W^l a^{l-1} + b^l$, the derivative with respect to the bias term is simply the identity matrix ($\frac{\partial z^l}{\partial b^l} = 1$) because b^l is added directly without any scaling factor. Substituting $\delta^l = \frac{\partial L}{\partial z^l}$:

$$\boxed{\frac{\partial L}{\partial b^l} = \delta^l}$$

This proves that the gradient with respect to any bias vector is identically equal to the local error vector of that layer.

2. Derivation of Hidden Layer Error Projection (BP2)

For a single neuron j in hidden layer l , its pre-activation z_j^l influences the loss L through **all** k neurons in the subsequent layer $l + 1$. By the multivariable chain rule:

$$\delta_j^l = \frac{\partial L}{\partial z_j^l} = \sum_k \frac{\partial L}{\partial z_k^{l+1}} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l}$$

By definition, the error at neuron k in layer $l + 1$ is $\delta_k^{l+1} = \frac{\partial L}{\partial z_k^{l+1}}$. Substituting this into the summation gives:

$$\delta_j^l = \sum_k \delta_k^{l+1} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l}$$

Next, we evaluate the internal derivative $\frac{\partial z_k^{l+1}}{\partial z_j^l}$. Knowing that $z_k^{l+1} = \sum_m W_{km}^{l+1} a_m^l + b_k^{l+1}$ and $a_j^l = \sigma(z_j^l)$:

$$\frac{\partial z_k^{l+1}}{\partial z_j^l} = \frac{\partial z_k^{l+1}}{\partial a_j^l} \cdot \frac{\partial a_j^l}{\partial z_j^l} = W_{kj}^{l+1} \cdot \sigma'(z_j^l)$$

Substituting this derivative back into the sum yields:

$$\delta_j^l = \sum_k \delta_k^{l+1} \cdot W_{kj}^{l+1} \cdot \sigma'(z_j^l) = \left(\sum_k W_{kj}^{l+1} \delta_k^{l+1} \right) \cdot \sigma'(z_j^l)$$

3.3 Vectorization and Matrix Transposition

The scalar sum $\sum_k W_{kj}^{l+1} \delta_k^{l+1}$ computes the inner product using the column index j of matrix W^{l+1} . To express this in standard matrix-vector multiplication notation, we must transpose the weight matrix $(W^{l+1})^T$.

Extending the scalar result across all neurons in layer l using the Hadamard product ($\odot$) for element-wise multiplication by activation derivatives:

$$\delta^l = \left((W^{l+1})^T \delta^{l+1} \right) \odot \sigma'(z^l)$$

This proves that $(W^{l+1})^T$ acts as the backward mapping mechanism, distributing downstream errors proportionally to the weights that propagated them forward.

3.4 Output Layer Gradient (δ^2)

For a Softmax output combined with categorical cross-entropy loss, the gradient with respect to the pre-activation values simplifies to

$$\delta^2 = A^2 - Y$$

Therefore,

$$\delta^2 \approx \begin{bmatrix} 0.4395 & 0.6381 \\ 0.5605 & 0.3619 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} -0.5605 & 0.6381 \\ 0.5605 & -0.6381 \end{bmatrix}$$

3.5 Weight and Bias Gradients at Layer 2 (dW^2, db^2)

The gradient with respect to the weights of Layer 2 is

$$dW^2 = \frac{1}{m} \delta^2 (A^1)^T$$

and the gradient with respect to the biases is

$$db^2 = \frac{1}{m} \sum_{i=1}^m \delta_i^2$$

Thus,

$$dW^2 = \frac{1}{2} \begin{bmatrix} -0.5605 & 0.6381 \\ 0.5605 & -0.6381 \end{bmatrix} \begin{bmatrix} 0.000 & 0.270 & 0.000 \\ 0.000 & 0.000 & 0.810 \end{bmatrix}$$

which gives approximately

$$\begin{aligned} dW^2 &\approx \frac{1}{2} \begin{bmatrix} 0.000 & -0.1513 & 0.5168 \\ 0.000 & 0.1513 & -0.5168 \end{bmatrix} \\ &= \begin{bmatrix} 0.0000 & -0.0757 & 0.2584 \\ 0.0000 & 0.0757 & -0.2584 \end{bmatrix} \end{aligned}$$

For the bias gradient,

$$db^2 = \frac{1}{2} \begin{bmatrix} -0.5605 + 0.6381 \\ 0.5605 - 0.6381 \end{bmatrix}$$

and therefore

$$db^2 \approx \begin{bmatrix} 0.0388 \\ -0.0388 \end{bmatrix}$$

3.6 Hidden Layer Error Projection (δ^1)

$$\delta^1 = [(W^2)^T \delta^2] \odot \text{ReLU}'(Z^1)$$

where

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases} \implies \text{ReLU}'(Z^1) = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

First, we compute the backprojected error:

$$(W^2)^T \delta^2 = \begin{bmatrix} 0.40 & -0.10 \\ -0.30 & 0.60 \\ 0.50 & -0.20 \end{bmatrix} \begin{bmatrix} -0.5605 & 0.6381 \\ 0.5605 & -0.6381 \end{bmatrix} \approx \begin{bmatrix} -0.2803 & 0.3191 \\ 0.5045 & -0.5743 \\ -0.3924 & 0.4467 \end{bmatrix}$$

Applying the Hadamard product with the ReLU derivative:

$$\delta^1 \approx \begin{bmatrix} -0.2803 & 0.3191 \\ 0.5045 & -0.5743 \\ -0.3924 & 0.4467 \end{bmatrix} \odot \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.0000 & 0.0000 \\ 0.5045 & 0.0000 \\ 0.0000 & 0.4467 \end{bmatrix}$$

3.7 Weight and Bias Gradients at Layer 1 (dW^1, db^1)

$$dW^1 = \frac{1}{m} \delta^1 (A^0)^T = \frac{1}{2} \begin{bmatrix} 0.0000 & 0.0000 \\ 0.5045 & 0.0000 \\ 0.0000 & 0.4467 \end{bmatrix} \begin{bmatrix} 0.5 & 0.8 & -0.1 & 0.3 \\ -0.2 & 0.4 & 0.9 & -0.6 \end{bmatrix}$$

$$dW^1 \approx \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.1261 & 0.2018 & -0.0252 & 0.0757 \\ -0.0447 & 0.0893 & 0.2010 & -0.1340 \end{bmatrix}$$

For the bias gradient:

$$db^1 = \frac{1}{m} \sum_{i=1}^m \delta_{:,i}^1 = \frac{1}{2} \begin{bmatrix} 0.0000 + 0.0000 \\ 0.5045 + 0.0000 \\ 0.0000 + 0.4467 \end{bmatrix} \approx \begin{bmatrix} 0.0000 \\ 0.2523 \\ 0.2234 \end{bmatrix}$$

4 Step 3: Adam Optimizer Step

Adam combines momentum-based optimization and adaptive learning rates. For each parameter θ , Adam maintains a first moment estimate m_t (mean of gradients) and a second raw moment estimate v_t (uncentered variance of gradients).

4.1 Accumulation & Bias Correction Formulas

The update rules for step t are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction accounts for initialization at zero ($m_0 = 0, v_0 = 0$):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

For the first iteration ($t = 1$) with standard hyper-parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$:

$$m_1 = 0.1 g_1 \implies \hat{m}_1 = \frac{0.1 g_1}{1 - 0.9} = g_1$$

$$v_1 = 0.001 g_1^2 \implies \hat{v}_1 = \frac{0.001 g_1^2}{1 - 0.999} = g_1^2$$

Therefore, at $t = 1$, the Adam parameter update simplifies to:

$$\theta_{\text{new}} = \theta - \alpha \frac{\hat{m}_1}{\sqrt{\hat{v}_1} + \epsilon} = \theta - \alpha \frac{g}{\sqrt{g^2} + \epsilon} = \theta - \alpha \frac{g}{|g| + \epsilon}$$

Given $\alpha = 0.001$ and $\epsilon = 10^{-8}$ (negligible for $g \neq 0$), the update rule simplifies element-wise to:

$$\theta_{\text{new}} \approx \theta - \alpha \cdot \text{sign}(g) \quad (g \neq 0), \quad \theta_{\text{new}} = \theta \quad (g = 0)$$

—

4.2 Explicit Parameter Updates

We now apply Adam explicitly to every weight matrix and bias vector.

4.2.1 Update of Layer 1 Weights (W^1)

Original Weights W^1 :

$$W^1 = \begin{bmatrix} 0.350 & -0.500 & 0.200 & -0.100 \\ -0.250 & 0.400 & -0.300 & 0.150 \\ 0.100 & 0.050 & 0.600 & -0.450 \end{bmatrix}$$

Gradients dW^1 :

$$dW^1 \approx \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.1261 & 0.2018 & -0.0252 & 0.0757 \\ -0.0447 & 0.0893 & 0.2010 & -0.1340 \end{bmatrix}$$

Element-wise sign of gradients $\text{sign}(dW^1)$:

$$\text{sign}(dW^1) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ +1 & +1 & -1 & +1 \\ -1 & +1 & +1 & -1 \end{bmatrix}$$

Applying $W_{\text{new}}^1 = W^1 - 0.001 \cdot \text{sign}(dW^1)$:

$$W_{\text{new}}^1 = \begin{bmatrix} 0.350 - 0.000 & -0.500 - 0.000 & 0.200 - 0.000 & -0.100 - 0.000 \\ -0.250 - 0.001 & 0.400 - 0.001 & -0.300 - (-0.001) & 0.150 - 0.001 \\ 0.100 - (-0.001) & 0.050 - 0.001 & 0.600 - 0.001 & -0.450 - (-0.001) \end{bmatrix}$$

$$W_{\text{new}}^1 = \begin{bmatrix} 0.350 & -0.500 & 0.200 & -0.100 \\ -0.251 & 0.399 & -0.299 & 0.149 \\ 0.101 & 0.049 & 0.599 & -0.449 \end{bmatrix}$$

—

4.2.2 Update of Layer 1 Biases (b^1)

Original Biases b^1 :

$$b^1 = \begin{bmatrix} 0.000 \\ 0.000 \\ 0.000 \end{bmatrix}$$

Gradients db^1 :

$$db^1 \approx \begin{bmatrix} 0.0000 \\ 0.2523 \\ 0.2234 \end{bmatrix} \implies \text{sign}(db^1) = \begin{bmatrix} 0 \\ +1 \\ +1 \end{bmatrix}$$

Applying $b_{\text{new}}^1 = b^1 - 0.001 \cdot \text{sign}(db^1)$:

$$b_{\text{new}}^1 = \begin{bmatrix} 0.000 - 0.000 \\ 0.000 - 0.001 \\ 0.000 - 0.001 \end{bmatrix} = \begin{bmatrix} 0.000 \\ -0.001 \\ -0.001 \end{bmatrix}$$

—

4.2.3 Update of Layer 2 Weights (W^2)

Original Weights W^2 :

$$W^2 = \begin{bmatrix} 0.400 & -0.300 & 0.500 \\ -0.100 & 0.600 & -0.200 \end{bmatrix}$$

Gradients dW^2 :

$$dW^2 \approx \begin{bmatrix} 0.0000 & -0.0757 & 0.2584 \\ 0.0000 & 0.0757 & -0.2584 \end{bmatrix} \implies \text{sign}(dW^2) = \begin{bmatrix} 0 & -1 & +1 \\ 0 & +1 & -1 \end{bmatrix}$$

Applying $W_{\text{new}}^2 = W^2 - 0.001 \cdot \text{sign}(dW^2)$:

$$W_{\text{new}}^2 = \begin{bmatrix} 0.400 - 0.000 & -0.300 - (-0.001) & 0.500 - 0.001 \\ -0.100 - 0.000 & 0.600 - 0.001 & -0.200 - (-0.001) \end{bmatrix} = \begin{bmatrix} 0.400 & -0.299 & 0.499 \\ -0.100 & 0.599 & -0.199 \end{bmatrix}$$

—

4.2.4 Update of Layer 2 Biases (b^2)

Original Biases b^2 :

$$b^2 = \begin{bmatrix} 0.000 \\ 0.000 \end{bmatrix}$$

Gradients db^2 :

$$db^2 \approx \begin{bmatrix} 0.0388 \\ -0.0388 \end{bmatrix} \implies \text{sign}(db^2) = \begin{bmatrix} +1 \\ -1 \end{bmatrix}$$

Applying $b_{\text{new}}^2 = b^2 - 0.001 \cdot \text{sign}(db^2)$:

$$b_{\text{new}}^2 = \begin{bmatrix} 0.000 - 0.001 \\ 0.000 - (-0.001) \end{bmatrix} = \begin{bmatrix} -0.001 \\ 0.001 \end{bmatrix}$$

—

4.3 Summary of Parameter Updates

These updated parameters constitute the exact input state for iteration $t = 2$:

$$W_{\text{new}}^1 = \begin{bmatrix} 0.350 & -0.500 & 0.200 & -0.100 \\ -0.251 & 0.399 & -0.299 & 0.149 \\ 0.101 & 0.049 & 0.599 & -0.449 \end{bmatrix}, \quad b_{\text{new}}^1 = \begin{bmatrix} 0.000 \\ -0.001 \\ -0.001 \end{bmatrix}$$

$$W_{\text{new}}^2 = \begin{bmatrix} 0.400 & -0.299 & 0.499 \\ -0.100 & 0.599 & -0.199 \end{bmatrix}, \quad b_{\text{new}}^2 = \begin{bmatrix} -0.001 \\ 0.001 \end{bmatrix}$$

4.4 Behavior in Subsequent Iterations ($t \gg 1$)

While $t = 1$ simplifies to a sign-based step ($\pm\alpha$) due to initial bias-correction scaling, Adam's true adaptive nature emerges as training progresses ($t > 1$):

1. **Decaying Bias Correction:** As step count t grows, the factor $(1 - \beta_2^t)$ rapidly approaches 1 (since $\beta_2^t \rightarrow 0$). The denominator transitions from scaling with current gradients to tracking the true exponential moving average of squared historical gradients:

$$\hat{v}_t \approx v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

2. **Adaptive Step Sizing via Denominator:** The term $\sqrt{\hat{v}_t} + \epsilon$ acts as an element-wise standard deviation normalizer:

- **Frequent/Large Gradients:** Produce large v_t values, resulting in a larger denominator. This automatically **scales down** the effective learning rate, preventing dramatic oscillation.
- **Infrequent/Sparse Gradients:** Produce small v_t values, resulting in a smaller denominator. This automatically **amplifies** updates, ensuring sparse parameters continue to learn effectively.

3. **Momentum Smoothing:** The numerator $\hat{m}_t$ accumulates past directional momentum ($m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$). This dampens high-frequency noise in mini-batch gradients and accelerates convergence through flat valleys where pure SGD slows down.

Thus, Adam seamlessly transitions from an aggressive sign-like exploratory step at initialization ($t = 1$) to a stable, curvature-aware adaptive optimization regime as historical statistics accumulate.